

Terminal Rewards Are Sufficient: Reasoning-Targeted Reward Decomposition Does Not Detectably Improve GRPO Training of a Small Debugging Model

Shashaank Jain · Pranav Pulipati

Code, dataset, per-bug results, and pre-registration: <https://github.com/PulipatiPranav/AgentDebuggerEnv>

Keywords: reinforcement learning, reward shaping, GRPO, program repair, verifiable rewards, negative results, evaluation methodology

Abstract

Dense, hand-crafted reward decompositions are widely assumed to help reinforcement learning of language-model reasoning: paying separately for a stated hypothesis, correct fault localization, and output format should give a weak policy gradient signal that a sparse pass/fail reward cannot. We test this assumption directly. In a pre-registered, leakage-controlled experiment, we train Qwen2.5-Coder-3B-Instruct with GRPO and LoRA to repair single-function Python bugs under three reward configurations that differ only in their component ceilings: R0, a seven-component dense reward that pays for hypothesis quality, localization, format, and similarity to the reference fix; R1, a terminal reward paying only for test outcomes and regressions; and R2, the dense reward with exactly the two reasoning-targeted components removed. Across three seeds per arm, evaluated greedily on 90 held-out mutation bugs, all three configurations are statistically indistinguishable (paired McNemar, all Holm-corrected $p = 1.0$ at the pooled level); point estimates slightly favor the *terminal* reward (48.5% vs. 45.6%), and a paired bootstrap bounds any dense-reward advantage below +2.6 points at 95% confidence — smaller than the training effect itself (+9.7 points over the zero-shot baseline, CI [+1.0, +18.5]). We additionally report a cautionary evaluation finding: an apparent 85-point deficit of free-form prompting was entirely a token-budget truncation artifact, and after correction the *untrained* free-form baseline matches the RL-trained structured policy. Reasoning-targeted reward engineering, at this scale and task, bought no measurable performance.

1. Introduction

Reinforcement learning from verifiable rewards has become the standard recipe for improving language-model performance on tasks with checkable outcomes, including mathematics and code [Lambert et al. 2024; Shao et al. 2024; Guo et al. 2025]. A recurring design question in these systems is *reward density*. The outcome signal — did the tests pass? — is sparse: for a weak policy it is almost always zero, and in group-relative methods such as GRPO a group whose completions all fail identically contributes exactly zero gradient. A natural response, common in practice, is to decompose the reward into hand-crafted components that pay for intermediate desiderata: emitting the required format, stating a specific hypothesis about the fault, naming the buggy function, resembling the reference solution. The implicit claim is that paying for the *reasoning* gives the policy something to climb before it can reliably produce passing fixes.

This claim is rarely tested in isolation. Comparisons in the literature typically change several things at once — the reward, the prompt format, the training data — so the marginal value of reasoning-targeted shaping is unknown. Process-supervision studies give conflicting guidance: process-based feedback can improve the faithfulness of reasoning traces [Lightman et al. 2023], yet outcome-only supervision often matches it on final-answer accuracy [Uesato et al. 2022], and large-scale RL systems have deliberately avoided learned process rewards because they invite reward hacking [Guo et al. 2025].

We test the claim directly, in a setting small enough to control completely. We built a debugging environment in which every candidate fix executes in a hardened sandbox against the bug's test cases, and a seven-component dense reward (R0) — hypothesis quality, localization, format compliance, fix quality, similarity to the reference fix, efficiency, and penalties — is implemented as a set of *component ceilings* on a single scoring class. An ablation is therefore a configuration, not a fork: R1 zeroes every shaping component and rescales the test-outcome term to the same range; R2 zeroes exactly the two reasoning-targeted components (hypothesis quality and localization) and keeps the rest. The three configurations were pre-registered, together with the hypotheses, statistical tests, and threats to validity, before any experiment ran.

On 90 held-out bugs — a fixed, committed, tier-stratified split with zero overlap with the 90 training bugs — the answer is unambiguous at our sample size: **the reward decomposition does not matter**. Training helps (all nine RL runs exceed the zero-shot baseline; pooled effect +9.7 points, 95% CI [+1.0, +18.5]), but the terminal-only reward matches or slightly exceeds the full dense reward, no pairwise comparison approaches significance after Holm correction, and the terminal and no-reasoning arms are *exactly* tied at the pooled level (28 discordant bugs in each direction over 270 paired observations).

Our contributions:

1. **A controlled negative result on reasoning-targeted reward decomposition.** Under GRPO at the 3B scale, a seven-component dense reward engineered to pay for debugging *reasoning* provides no detectable benefit over a terminal pass/fail-with-penalties reward; the paired bootstrap bounds any dense-reward advantage below +2.6 points, roughly a quarter of the training effect itself. The exact tie between R1 and R2 further shows that even non-reasoning dense shaping (format, similarity, efficiency) adds nothing here.
2. **A leakage-controlled, execution-validated testbed for RL on debugging.** 180 single-function Python bugs generated by tiered AST mutations of MBPP programs, each validated by execution (reference passes all tests, mutant fails at least one), with a committed train/held-out split, a hardened execution sandbox, and a single scoring path shared verbatim by training and evaluation. The core package has no third-party dependencies.
3. **A cautionary evaluation-methodology finding.** Our initial free-form baseline scored 1.1% with an 86.7% extraction-failure rate — apparently a dramatic argument for structured output. The entire gap was a hard-coded 300-token generation budget that truncated free-form responses before any code appeared. At 700 tokens, the *untrained* free-form baseline solves 45.6%, exactly matching the mean of the RL-trained structured arm. Evaluation-harness artifacts can fabricate effects an order of magnitude larger than the real ones under study.

4. **A pre-registered design with reported deviations.** We state which pre-registered hypotheses we could and could not test, and why, rather than reporting only the comparisons that ran.

2. Related work

RL for code generation and repair. CodeRL introduced actor-critic RL over unit-test feedback for program synthesis [Le et al. 2022]; subsequent work established RL with verifiable rewards (RLVR) as a general post-training recipe [Lambert et al. 2024]. GRPO [Shao et al. 2024] replaces the learned value function with group-relative advantages over sampled completions, and is the algorithm behind DeepSeek-R1's reasoning training [Guo et al. 2025]. Notably, DeepSeek-R1 used only rule-based *outcome* rewards plus a format term, explicitly rejecting neural process rewards as hack-prone; our result is a controlled, small-scale complement to that design decision — even *rule-based* reasoning shaping bought nothing in our setting.

Process vs. outcome supervision. Uesato et al. [2022] found that outcome-based and process-based feedback reach similar final-answer accuracy on GSM8K, while process supervision improves the reasoning traces themselves; Lightman et al. [2023] found process reward models superior for best-of-N selection on harder mathematics. Our experiment addresses a different point in this space: not learned reward *models*, but hand-crafted heuristic shaping terms inside the RL reward, evaluated by final task success. Our finding aligns with the outcome-sufficiency side of this literature.

Reward shaping. Ng et al. [1999] characterized the potential-based shaping functions that leave optimal policies invariant. Our dense components are not potential-based — they pay for properties of the response text itself — so they can in principle change the learned policy; empirically, they did not change what it achieves.

Debugging benchmarks. QuixBugs [Lin et al. 2017] and DebugBench [Tian et al. 2024] evaluate LLM debugging on single functions; SWE-bench [Jimenez et al. 2024] evaluates repository-level repair; Self-Debugging prompts models to critique their own code [Chen et al. 2024]. These are evaluation suites; our environment is a *training* environment with a reward, curriculum, and sandbox, and its bugs are freshly generated mutations rather than published problems, reducing contamination. We adapted 27 QuixBugs programs as a transfer set but did not evaluate on it (§8).

Structured output constraints. Tam et al. [2024] report that format restrictions can degrade LLM reasoning performance. Our corrected baselines are consistent with a related point: the zero-shot model performs *better* free-form than under a strict five-field schema (45.6% vs. 37.8%, not significant at $n=90$), and the apparent catastrophic failure of free-form output was an artifact of the evaluation harness, not the format.

Curriculum learning [Bengio et al. 2009] motivates our tiered schedule; we emphasize that our curriculum's premise did not survive contact with data (§7.3) and no curriculum claim is made.

3. The environment

3.1 Task and dataset

Each task instance is a single Python function containing exactly one injected bug, with 3–6 input/expected-output test cases and an initial failing-test message. The agent sees the buggy source and the initial error; it must return a response from which a complete replacement function is extracted, executed in the sandbox against all test cases, and scored.

The dataset contains 180 bugs generated from MBPP [Austin et al. 2021] reference solutions by single AST mutations, in three tiers by mutation-operator category: tier 1, boundary/off-by-one (comparison-boundary flips, integer-constant ± 1); tier 2, wrong operator/logic (arithmetic and boolean operator swaps, equality flips, comparison reversal); tier 3, edge-case (slice/range-bound tweaks, removed base-case guards). Problems whose expected outputs are not exactly JSON-round-trippable (floats, tuples, sets) or that depend on nondeterministic modules are excluded, so the sandbox's exact-equality check is sound. Every record is validated by execution: the reference solution passes all its cases and the mutant fails at least one. Tier labels describe the mutation operator, **not** measured difficulty — a distinction that turns out to matter (§7.3).

The 180 bugs are split 90/90 into train and held-out sets, stratified by tier (30 per tier per side), by a committed, immutable list of bug ids. All training draws only from the train side; every number reported in this paper is computed on the held-out side. We verified programmatically that all twelve published evaluation files score exactly the 90 held-out ids and none of the train ids.

Two checks calibrate difficulty. The reference (oracle) fix passes by construction. Zero-shot Llama-3.3-70B solves 67.8% of the held-out set — the environment is neither saturated by a much larger model nor trivially easy, leaving a real gap for training to close at 3B.

3.2 Sandbox

Model-generated code executes in a three-layer sandbox: (i) static AST analysis in the parent process refuses blocked imports, builtins (`eval`, `exec`, `open`, `getattr`, ...) and dunder-attribute escapes before any code runs; (ii) kernel `setrlimit` ceilings on address space (256 MB), CPU time (10 s), and file writes (0 bytes) in the child; (iii) a 10 s wall-clock deadline that kills the child's whole process group. The `stdlib` a legitimate fix needs (`hashlib`, `threading`, `super()`) is unaffected. Fifteen named escape attempts are covered by tests. Training, evaluation, and dataset validation all execute through this one path, so no two components can disagree about what a passing test means.

3.3 Response formats

Structured (the trained format): the model must emit five labeled fields — `OBSERVATION`, `HYPOTHESIS`, `CONFIDENCE` (low/medium/high), `ACTION`, `DETAIL` — with the fix as a fenced code block in `DETAIL`. Parsing is whitespace/case-tolerant and never raises; malformed responses are marked invalid and priced by the reward.

Free-form: no schema; a length-matched system prompt (within 15% by word count, enforced by test, to control the prompt-length confound) shows a worked example of prose reasoning followed by one fenced code block. The extractor takes the *last* fenced block, falling back to the whole

response, and reports an extraction-failure rate that is directly comparable to the structured arm's format-failure rate.

3.4 Reward configurations

The trained setting is single-turn, so the RL problem is a contextual bandit: the state is the bug prompt x , the action is the full sampled completion y , and the reward is $r(x, y)$ computed by one scoring function shared by training and evaluation (asserted by test). The shipped reward R_0 decomposes as

$$r(x,y) = f_{\text{format}} + f_{\text{hyp}} + f_{\text{loc}} + f_{\text{fix}} + f_{\text{sem}} + f_{\text{eff}} + f_{\text{pen}}, \text{ clipped below at } -0.5,$$

with component ceilings:

Component	Ceiling	Pays for	R0	R1	R2
format_compliance	0.10	the five required fields (partial credit per field)	✓	—	✓
hypothesis_quality	0.20	specific, grounded, confidence-calibrated hypothesis	✓	—	—
localization	0.15	naming the buggy function (+0.08) and line (+0.07)	✓	—	—
fix_quality	0.35	graded test pass rate; full ceiling only at 100%	✓	✓ (→ 1.0)	✓
semantic_similarity	0.10	edit similarity to the reference fix	✓	—	✓
efficiency_potential	0.10	0.02 per remaining turn	✓	—	✓
penalties	−0.55	regressions (−0.20), give-up (−0.15), invalid/malformed (−0.10 each)	✓	✓	✓

A perfect first-turn solve scores exactly 1.0 and the floor is −0.5 (both asserted by tests). `hypothesis_quality` rewards `length ≥ 20` words, citing code symbols and numbers, lexical overlap with the stated observation, and confidence calibration (a high-confidence wrong fix is penalized relative to a cautious one); it is a *heuristic over the response text*, not an LLM judge. A bug counts as **solved** iff every test case passes.

R1 (terminal) zeroes every shaping component and rescales `fix_quality` to a 1.0 ceiling (preserving its graded shape), keeping penalties — the reward an ordinary outcome-driven RLVR setup would use, occupying the same [−0.5, 1.0] range as R_0 so reward magnitude is not a confound. **R2 (no-reasoning)** zeroes exactly `hypothesis_quality` and `localization`, leaving a still-dense reward (max 0.65; GRPO's group normalization makes the uniform scale largely irrelevant, and we deliberately do not rescale to avoid perturbing relative component weights). R_0 vs. R_1 tests whether decomposition helps at all; R_0 vs. R_2 tests whether the *reasoning* components specifically help; R_1 vs. R_2 tests whether any dense shaping helps.

3.5 Training

We train Qwen2.5-Coder-3B-Instruct [Hui et al. 2024] with GRPO [Shao et al. 2024] and LoRA [Hu et al. 2022] via TRL. Batch geometry is selected deterministically from detected VRAM; the runs reported here used 24 GB-class GPUs (RTX 4090), giving per-device batch 2, gradient accumulation 4, **group size (num_generations) 2**, completion budget 192 tokens (structured), LoRA rank 8 ($\alpha=16$, all attention and MLP projections). Learning rate $2e-5$ with cosine decay and 30 warmup steps (first stage only), sampling temperature 0.7, 500 optimizer steps, seeds {42, 123, 456} controlling initialization, sampling, and data order. A tiered curriculum unlocks tier 2 at step 150 and tier 3 at

step 350; because TRL's sampler caches the dataset at trainer construction, each curriculum stage runs a fresh GRPOTrainer carrying the LoRA adapter forward — without this, the bug pool silently never grows (a failure mode we document because it invalidates naive dataset-swapping callbacks). A completion whose scoring crashes receives -0.3 . The small group size is a stated limitation (§8): with two completions per group, the group-relative advantage is a sign, and dense rewards' ability to break ties inside a group is used at its weakest.

4. Experimental setup

Arms. All arms share the base model, prompts, curriculum, optimizer, step budget, and evaluation protocol; only the factor under test varies.

Arm	Training	Reward	Format	Seeds
B0	none (zero-shot)	—	structured	1 (greedy)
B1	none (zero-shot)	—	free-form	1 (greedy)
E1	GRPO	R0 full	structured	3
E3	GRPO	R1 terminal	structured	3
E4	GRPO	R2 no-reasoning	structured	3
E2	GRPO	R1	free-form	dropped (§4.1)

Evaluation. Greedy decoding (an evaluation that changes its answer between runs cannot be compared), one response per bug, 90 held-out bugs, format-aware generation budget (300 tokens structured / 700 free-form, §7.4), truncation detection logged per record. Primary metric: held-out solve rate (all tests pass). Secondary: per-tier solve rate, mean reward, extraction-failure rate.

Statistics. All comparisons are paired on the bug (every arm sees the identical held-out set). Primary test: exact-binomial McNemar on discordant pairs, Holm–Bonferroni-corrected within each comparison family, at three levels: per-seed (9 tests), majority-vote pooling (a bug counts as solved if $\geq 2/3$ seeds solve it), and all-pairs (each bug $\times$ seed as one paired observation, $N=270$; observations sharing a bug are dependent, so this level is reported as corroborating, not primary). Wilson 95% intervals on all solve rates. Paired bootstrap (10,000 resamples over bugs, seed-averaged solve indicator per bug) for difference CIs. **Minimum detectable effect:** at $\alpha=0.05$ and 80% power, 90 paired bugs detect differences of roughly 16 points; smaller true effects will read as "not detected," and we never report them as "no effect." All statistics in this paper were recomputed independently from the committed per-bug records.

4.1 Deviations from the pre-registration

The pre-registered design [research_plan.md in the repository] specified three hypotheses at two model scales. We report what actually happened. (i) **H1 (structured vs. free-form under equal reward) was not tested:** the free-form RL arm (E2) requires a 550-token completion budget, making each GRPO step $\sim 2\times$ slower (~ 90 s vs. ~ 40 s); the three-seed protocol was compute-infeasible on our hardware. The corrected B1 baseline provides observational evidence only. (ii) **H3 (curriculum vs. flat sampling) was not run;** the pre-registration itself flagged it as underpowered at 3 seeds. All arms share the same curriculum, so it is a controlled constant, not a tested claim. (iii) The 7B scale and the

GPT-4o-mini calibration baseline were replaced, for budget reasons, by the 3B scale and a zero-shot Llama-3.3-70B difficulty gate. (iv) H2's prediction was **directionally wrong**: the pre-registration predicted $R0 > R1$; the data show $R1 \geq R0$.

5. Results

Table 1: held-out solve rate (90 bugs; Wilson 95% CI for single runs; per-seed values and mean for RL arms).

Arm	Overall	Tier 1	Tier 2	Tier 3
B0 zero-shot, structured	37.8% [28.5, 48.1]	30.0%	43.3%	40.0%
B1 zero-shot, free-form (corrected)	45.6% [35.7, 55.8]	40.0%	36.7%	60.0%
E1 (R0 full): 47.8 / 46.7 / 42.2	45.6% mean	34.4%	45.6%	56.7%
E3 (R1 terminal): 41.1 / 51.1 / 53.3	48.5% mean	33.3%	55.6%	56.7%
E4 (R2 no-reasoning): 52.2 / 48.9 / 44.4	48.5% mean	34.4%	50.0%	61.1%
Llama-3.3-70B zero-shot (gate)	67.8%	66.7%	56.7%	80.0%

Extraction-failure rate is 0.0% in every arm above. All 90 responses in every RL run were propose_fix actions; no run produced a fix that broke a previously passing test (newly_broken = 0 throughout).

RQ1 — Does GRPO training improve held-out debugging? Yes, modestly. Every one of the nine RL runs exceeds B0's 37.8%. Pooling all nine runs, the paired bootstrap gives +9.7 points over B0, 95% CI [+1.0, +18.5]. Per arm: E3–B0 = +10.7 [+1.5, +20.0] and E4–B0 = +10.9 [+1.1, +20.4] exclude zero; E1–B0 = +7.7 [−0.7, +16.3] does not. The improvement is concentrated on tiers 2–3 (e.g., E1 tier 3: 56.7% vs. B0's 40.0%); tier-1 performance barely moves.

RQ2 — Does dense reward decomposition beat terminal reward? No detectable difference, and the point estimate goes the wrong way. E1 (R0) averages 45.6%; E3 (R1) averages 48.5%. Of nine per-seed McNemar tests across the three comparisons, none survives Holm correction (the two raw $p < 0.05$ — E1 vs. E3 seed 456, $p=0.041$, and E3 vs. E4 seed 42, $p=0.031$ — adjust to 0.33 and 0.28, and both favor the *less* shaped arm). At the majority-vote level: E1 vs. E3 discordants 8/10, $p=0.81$. The paired bootstrap CI on E1–E3 is [−8.5, +2.6]: under bug-resampling, any true advantage of the seven-component reward is at most +2.6 points at 95% confidence — about a quarter of the training effect. E1 vs. E4 is essentially identical ([−8.5, +2.6]).

RQ3 — Does any dense shaping beat terminal reward? No. E3 (terminal) and E4 (dense minus reasoning) both average 48.5%; at the all-pairs level they are *exactly* tied — 28 bugs solved only by E3, 28 only by E4 ($p=1.0$); bootstrap CI on the difference [−5.6, +5.6]. Whatever the format/similarity/efficiency components pay for, it does not show up in solve rate — and combined with RQ2, neither do the reasoning components.

RQ4 — How much damage can an evaluation artifact do? The original free-form baseline scored 1.1% (1/90) with 86.7% extraction failure — on its face, strong evidence that unstructured output is unusable. The root cause was a single hard-coded `max_new_tokens=300` in the evaluation generator: free-form responses write prose before code, and every failed completion was truncated mid-sentence before any code block appeared (the training path already contained a fix for this exact

symptom that had not been ported to evaluation). With a 700-token budget, B1 solves 45.6% with 0% extraction failure and 0/90 truncated completions — statistically indistinguishable from, and numerically equal to, the RL-trained structured mean (E1–B1 bootstrap CI $[-8.9, +9.3]$). A one-line harness bug manufactured an 85-point effect; the real format story at this scale is "no detectable difference, possibly favoring free-form" (B1–B0 = +7.8 $[-4.4, +20.0]$), directionally consistent with reports that format restrictions can hurt reasoning [Tam et al. 2024].

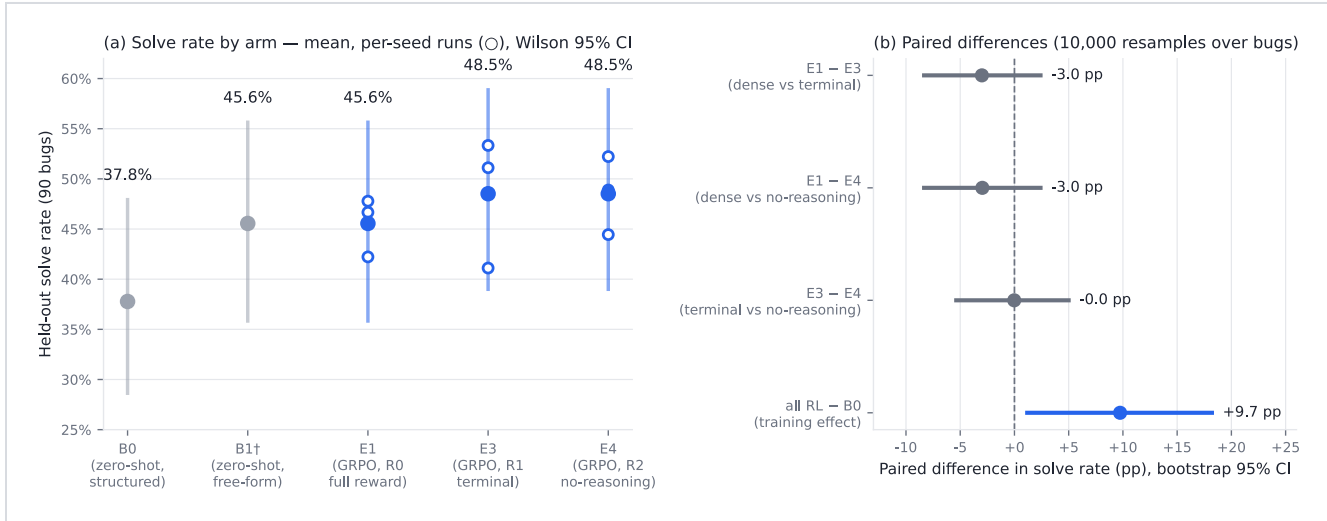


Figure 1. (a) Held-out solve rate by arm: arm mean (filled), per-seed runs (open circles), Wilson 95% CI at $n=90$. †B1 is the corrected 700-token free-form baseline. (b) Paired differences in solve rate with bootstrap 95% confidence intervals (10,000 resamples over bugs, seed-averaged per bug). The three reward-configuration contrasts straddle zero; the pooled training effect does not.

6. Analysis

Seed variance. The terminal-reward arm is noisier across seeds: E3's range is 12.2 points (41.1–53.3, sd 6.5) against E1's 5.6 (sd 2.9) and E4's 7.8 (sd 3.9). This is consistent with the pre-registered mechanism — sparser rewards produce more zero-variance GRPO groups and thus noisier gradients — but with three seeds per arm it is a descriptive observation, not a tested claim; the per-step degenerate-group fractions were logged to run telemetry but are not part of the committed record, so we do not analyze them here. The practical reading: decomposition may buy run-to-run *stability* rather than mean performance, a hypothesis our data can suggest but not confirm.

What stays unsolved. 20 of 90 held-out bugs are solved by no configuration — not by B0 nor by any of the nine RL runs — while 16 are solved by all nine. The hard core is dominated by boundary mutations (off-by-one and comparison-boundary flips account for 13 of the 20), and 11 of the 20 are tier-1 bugs. Single-character boundary errors are precisely the mutations whose buggy and correct versions are most similar as text, giving both the model and any similarity-based reward term the least signal.

Tier labels do not track difficulty. The pre-registration required verifying, before making any curriculum claim, that the base model finds tier 1 easier than tier 3. It does not: B0 solves 30% of tier 1, 43% of tier 2, and 40% of tier 3, and Llama-3.3-70B shows the same non-monotonicity (67/57/80). "Off-by-one" mutations are *categorically simple* but empirically the hardest to spot. Two

consequences: the curriculum trained easiest-labeled-first but not easiest-first, and no claim about curriculum benefit is made or supportable from this data. RL gains concentrating on tiers 2–3 — whose operator flips (+ → -, and → or) produce more conspicuous failures — is consistent with training teaching the model to exploit conspicuous symptom–cause links rather than to find subtle boundary errors.

Why might terminal reward suffice here? Three observations. First, the base policy is not weak enough to need shaping: at 37.8% zero-shot, roughly one completion in three earns nonzero terminal reward, so GRPO groups are frequently non-degenerate even under R1 — the regime where dense shaping should matter most (near-zero base success) is not the regime we are in. Second, the shaping components are heuristics over response *text* (word counts, symbol citations, lexical overlap, edit similarity), only weakly coupled to what makes a fix pass; gradient spent on them is not obviously spent on solving. Third, with group size 2 the advantage is effectively a sign bit, compressing the resolution at which a dense reward can rank completions — decomposition might matter more at larger group sizes, which we flag as the most direct follow-up.

7. Discussion

The negative result is practically useful. The seven-component reward is roughly four hundred lines of scoring heuristics, each a design decision that can be wrong, each a potential reward-hacking surface, each something a reviewer must audit. The terminal configuration is "graded pass rate plus penalties" — no hypothesis-quality heuristic, no localization matcher, no similarity metric — and it matched or beat the full reward while occupying the same range. For practitioners building RLVR pipelines for code at small scale, the actionable statement is: *before* investing in reasoning-targeted reward engineering, establish that the simple verifiable reward is insufficient, because at least in this controlled setting it was not.

The result is consistent with, and adds a controlled data point to, several strands of prior evidence: outcome-based supervision matching process-based on final accuracy [Uesato et al. 2022]; DeepSeek-R1's deliberate use of rule-based outcome-plus-format rewards over process reward models [Guo et al. 2025]; and the general RLVR recipe [Lambert et al. 2024]. What our experiment adds is isolation: same model, same data, same optimizer, same step budget, same prompts, same evaluation, with the reward configuration as the only moving part, and a pre-registered analysis plan.

Equally important is what the result does *not* license. It does not say dense rewards never help: our base model already solves a third of the tasks zero-shot; with a genuinely weak base policy, terminal reward may provide no gradient at all, and shaping (or curricula) may be necessary rather than decorative. It does not speak to multi-step agentic debugging, where intermediate actions have no test outcome and some density seems unavoidable. It does not rule out effects smaller than our minimum detectable effect (§8). And the truncation episode (RQ4) cuts both ways: it is a warning that some published effects of this kind may be harness artifacts, and a reminder that ours could have been too — which is why every number here is recomputed from committed per-bug records and the extraction-failure and truncation rates are reported alongside every result.

8. Limitations and ethics

Statistical power. 90 paired bugs detect ~16-point effects; a true 5-point benefit of dense reward would be invisible to our tests. Our claim is calibrated accordingly: *no detectable* difference, with a +2.6-point bootstrap upper bound on the dense advantage under bug-resampling. The bootstrap resamples bugs, not seeds; with three seeds per arm, seed-level uncertainty is underrepresented, and the E1–B0 interval's dependence on pooling illustrates the fragility.

Scale and family. One model (Qwen2.5-Coder-3B-Instruct), one algorithm (GRPO), one LoRA configuration, group size 2. The pre-registered 7B replication was not run. Conclusions should not be generalized beyond small-scale GRPO on this task family without replication; group size is a specific, testable moderator.

Task realism. Single-function, single-mutation bugs with exact-output tests are far from repository-scale debugging [Jimenez et al. 2024]. The 27-program QuixBugs transfer set was prepared but not evaluated; no transfer claim is made. Mutation-generated bugs also skew toward operator-level errors; the dataset's own tier labels failing to track difficulty (§6) shows how unreliable intuitions about synthetic bug difficulty are.

Untested pre-registered hypotheses. The structure hypothesis (H1) and curriculum hypothesis (H3) were not tested (§4.1); the curriculum is an uncontrolled constant shared by all arms, and its schedule was tuned for a difficulty ordering the data contradict.

Ethics. No human subjects or personal data are involved. Executing model-generated code is a real hazard; our sandbox (static analysis, kernel rlimits, process-group deadlines) is a workstation-grade defence and is documented as such, not as a substitute for VM isolation against deliberately adversarial code. MBPP is used under its open license with provenance recorded in the datacard. Total compute was modest (nine 500-step LoRA runs on 24 GB GPUs, ~40 s/step $\approx$ 50 GPU-hours plus evaluation), which we note deliberately: negative results at small scale are cheap to replicate, and we release everything needed to do so.

9. Conclusion

In a pre-registered, leakage-controlled comparison at the 3B scale, decomposing an RL debugging reward into seven components — including terms crafted specifically to pay for hypothesis formation and fault localization — produced no detectable improvement over a terminal pass/fail-with-penalties reward: point estimates favored the terminal arm, no comparison survived correction, the terminal and shaping-without-reasoning arms tied exactly, and any dense-reward advantage is bounded below +2.6 points at 95% confidence under bug-resampling. Training itself helped (+9.7 points pooled). A separate one-line evaluation bug briefly manufactured an 85-point format effect, dwarfing every real effect in the study. Both findings argue the same way: in small-scale RLVR for code, the expensive part is not the reward engineering — it is the experimental control needed to know whether the reward engineering did anything.

References

- Austin, J., Odena, A., Nye, M., et al. (2021). Program Synthesis with Large Language Models. arXiv:2108.07732.
- Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum Learning. *ICML 2009*.
- Chen, X., Lin, M., Schärli, N., & Zhou, D. (2024). Teaching Large Language Models to Self-Debug. *ICLR 2024*. arXiv:2304.05128.
- Guo, D., Yang, D., Zhang, H., et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.
- Holm, S. (1979). A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.
- Hu, E. J., Shen, Y., Wallis, P., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*. arXiv:2106.09685.
- Hui, B., Yang, J., Cui, Z., et al. (2024). Qwen2.5-Coder Technical Report. arXiv:2409.12186.
- Jimenez, C. E., Yang, J., Wettig, A., et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR 2024*. arXiv:2310.06770.
- Lambert, N., Morrison, J., Pyatkin, V., et al. (2024). Tülu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124.
- Le, H., Wang, Y., Gotmare, A. D., Savarese, S., & Hoi, S. C. H. (2022). CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. *NeurIPS 2022*. arXiv:2207.01780.
- Lightman, H., Kosaraju, V., Burda, Y., et al. (2023). Let's Verify Step by Step. *ICLR 2024*. arXiv:2305.20050.
- Lin, D., Koppel, J., Chen, A., & Solar-Lezama, A. (2017). QuixBugs: A Multi-Lingual Program Repair Benchmark Set Based on the Quixey Challenge. *SPLASH Companion 2017*.
- McNemar, Q. (1947). Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2), 153–157.
- Ng, A. Y., Harada, D., & Russell, S. (1999). Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. *ICML 1999*.
- Shao, Z., Wang, P., Zhu, Q., et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.
- Tam, Z. R., Wu, C.-K., Tsai, Y.-L., Lin, C.-Y., Lee, H., & Chen, Y.-N. (2024). Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models. *EMNLP 2024 Industry Track*. arXiv:2408.02442.
- Tian, R., Ye, Y., Qin, Y., et al. (2024). DebugBench: Evaluating Debugging Capability of Large Language Models. *Findings of ACL 2024*. arXiv:2401.04621.
- Uesato, J., Kushman, N., Kumar, R., et al. (2022). Solving Math Word Problems with Process- and Outcome-Based Feedback. arXiv:2211.14275.